
Solução de Sistemas de Equações Lineares

Trabalho 1

Leonardo Peres Albertazzi Drummond

Marco Antonio Tronco Felix

Eduardo de Mendonça Freire

João Pedro Peçanha Cotrim

Carlos Augusto Bertão Rodrigues

Andre Pacífico

1º Semestre de 2026

Resumo

Este relatório descreve a implementação, validação e análise de desempenho de cinco métodos para solução de sistemas lineares quadrados $Ax = b$: Eliminação Gaussiana, Fatoração LU, Decomposição de Cholesky, método de Jacobi e método de Gauss-Seidel. Os métodos foram aplicados a doze matrizes esparsas do repositório Matrix Market, provenientes de problemas de engenharia estrutural, sistemas de potência e engenharia elétrica. Os resultados confirmam que (i) Cholesky requer aproximadamente metade do custo computacional de Gauss/LU, em acordo com a teoria; (ii) os métodos iterativos, especialmente o método de Jacobi, falham em algumas das matrizes por mal condicionamento e ausência de dominância diagonal; e (iii) as implementações em Python puro alcançam a mesma precisão numérica que as bibliotecas NumPy/SciPy, porém com velocidade até $632\times$ inferior à das bibliotecas.

Sumário

1	Introdução	3
2	Descrição dos métodos	3
2.1	Métodos diretos	3
2.1.1	Eliminação Gaussiana	3
2.1.2	Fatoração LU	3
2.1.3	Decomposição de Cholesky	4
2.2	Métodos iterativos	4
2.2.1	Método de Jacobi	4
2.2.2	Método de Gauss-Seidel	4
2.2.3	Critério de parada	4
2.3	Resumo comparativo	4
3	Implementação	5
3.1	Estrutura do projeto	5
3.2	Núcleo das implementações	5
3.3	Verificação, logging, saída, Teste	6
4	Matrizes de teste	6
4.1	Matrizes Mal-Condicionadas	6
4.2	Matrizes Bem-Condicionadas	7
5	Resultados	7
5.1	Matrizes Mal-Condicionadas	7
5.1.1	Métodos diretos	7
5.1.2	Métodos iterativos	8
5.1.3	Gráficos $\log(r)$ x iterações	8
5.2	Matrizes Bem-Condicionadas	15
5.2.1	Métodos diretos	15
5.2.2	Métodos iterativos	16
5.2.3	Gráficos $\log(r)$ x iterações	16
6	Comparação: implementação pura vs. bibliotecas	21
6.1	Tempo de execução	21
6.2	Precisão numérica	21
7	Validação da teoria: razão Cholesky/Gauss	22
8	Conclusões	22
	Reprodutibilidade	22

1 Introdução

Resolver sistemas lineares da forma

$$Ax = b, \quad A \in \mathbb{R}^{n \times n}, \quad (1)$$

é uma tarefa central em praticamente todas as áreas da engenharia e das ciências computacionais. Em problemas reais — engenharia estrutural, redes elétricas, simulação de equações diferenciais parciais — a dimensão n pode variar de centenas a milhões, tornando a escolha do método de resolução crítica tanto em termos de custo quanto de precisão.

Este trabalho implementa e compara dois grandes paradigmas de solução:

Métodos diretos

Fatoram A em uma forma triangular ou produto de matrizes. O resultado é obtido em um único passo (fatoração + substituição) e é exato a menos de erros de arredondamento em ponto flutuante.

Métodos iterativos

Partem de uma estimativa inicial $x^{(0)}$ e refinam progressivamente a solução até satisfazer um critério de parada baseado em tolerância ou número máximo de iterações.

O objetivo é estudar empiricamente a eficiência, a precisão e as condições de convergência de cada abordagem sobre matrizes de referência com características distintas.

2 Descrição dos métodos

2.1 Métodos diretos

2.1.1 Eliminação Gaussiana

A Eliminação Gaussiana transforma A em uma matriz triangular superior U por meio de combinações lineares de linhas. Em seguida, o sistema $Ux = b'$ é resolvido por retrossubstituição. O custo total é de aproximadamente $\frac{2n^3}{3}$ operações de ponto flutuante, sendo aplicável a qualquer matriz não-singular.

2.1.2 Fatoração LU

A Fatoração LU aplica a mesma ideia da Eliminação Gaussiana, mas guarda explicitamente a fatoração $A = LU$, onde L é triangular inferior com diagonal unitária e U é triangular superior. O custo de fatoração é idêntico ($\approx \frac{2n^3}{3}$), mas a vantagem é significativa quando se precisa resolver o sistema para múltiplos vetores b : após a fatoração, cada novo sistema custa apenas $\mathcal{O}(n^2)$, sem refatorar.

2.1.3 Decomposição de Cholesky

Para matrizes simétricas positivas definidas (SPD), a Decomposição de Cholesky fatoriza $A = LL^T$, onde L é triangular inferior. As fórmulas de atualização são:

$$\ell_{ii} = \sqrt{a_{ii} - \sum_{k<i} \ell_{ik}^2}, \quad (2)$$

$$\ell_{ji} = \frac{a_{ji} - \sum_{k<i} \ell_{jk} \ell_{ik}}{\ell_{ii}}, \quad j > i. \quad (3)$$

O custo é de $\approx \frac{n^3}{3}$ operações, exatamente a metade de Gauss/LU, tornando-o o método direto mais eficiente para matrizes SPD.

2.2 Métodos iterativos

2.2.1 Método de Jacobi

O método de Jacobi atualiza todas as coordenadas em paralelo usando apenas os valores da iteração anterior:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}. \quad (4)$$

2.2.2 Método de Gauss-Seidel

O método de Gauss-Seidel melhora o Jacobi ao reaproveitar, na mesma iteração, os componentes já atualizados:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}}{a_{ii}}. \quad (5)$$

Em geral, Gauss-Seidel converge em menos iterações do que Jacobi.

2.2.3 Critério de parada

Ambos os métodos iterativos utilizam o critério de parada relativo:

$$R^{(k)} = \frac{\|x^{(k+1)} - x^{(k)}\|_2}{\|x^{(k+1)}\|_2} \leq t, \quad (6)$$

2.3 Resumo comparativo

A Tabela 1 resume as características de cada método.

Tabela 1: Comparação entre os métodos implementados.

Método	Custo	Requisito em A	Convergência	Quando usar
Gauss	$2n^3/3$	não-singular	garantida	qualquer caso
LU	$2n^3/3$	não-singular	garantida	múltiplos b
Cholesky	$n^3/3$	SPD	garantida	A SPD — $2\times$ mais rápido
Jacobi	n^2/iter	diag. dominante	condicional	paralelização
Gauss-Seidel	n^2/iter	diag. dominante	condicional	memória limitada

3 Implementação

3.1 Estrutura do projeto

O código foi organizado em módulos com responsabilidades bem definidas:

```

Trab1_Alglin_Comp/
main.py           # loop sobre as 6 matrizes
sistema_linear.py # classe SistemaLinear (todos os métodos)
testes.py         # orquestra: executa, mede, escreve
Dependencias/
  utilidades.py   # vetores, ler_matriz_market, ...
  log.py          # Log(iteração, resíduo, x)
  results.py      # escreve .txt
  sumario.py      # tabela comparativa em Markdown

```

A classe `SistemaLinear` encapsula a matriz A , o vetor b e a solução x . Cada método altera `self.x` e possui também uma versão com biblioteca para comparação (`np.linalg.solve`, `scipy.linalg.lu_factor`, `scipy.linalg.cho_factor`).

3.2 Núcleo das implementações

Fatoração LU. O fragmento abaixo ilustra o núcleo da fatoração em Python puro, sem uso de bibliotecas numéricas:

```

1 for i in range(n):
2     for j in range(i + 1, n):
3         mult = U[j][i] / U[i][i]
4         L[j][i] = mult
5         for k in range(i, n):
6             U[j][k] -= mult * U[i][k]

```

Listing 1: Núcleo da Fatoração LU (Python puro).

Decomposição de Cholesky.

```

1 for i in range(n):
2     soma_diag = self.A[i][i] - sum(L[i][k]**2 for k in range(i))
3     if soma_diag <= 0:
4         return "Matriz n o      sim trica positiva definida"
5     L[i][i] = soma_diag ** 0.5

```

```

6     for j in range(i + 1, n):
7         soma = self.A[j][i] - sum(L[j][k]*L[i][k] for k in range(i))
8         L[j][i] = soma / L[i][i]

```

Listing 2: Núcleo da Decomposição de Cholesky (Python puro).

Os métodos iterativos (Jacobi e Gauss-Seidel) seguem diretamente as equações (4) e (5), sem uso de bibliotecas.

3.3 Verificação, logging, saída, Teste

- **Verificação de correção:** após obtenção de x , calcula-se $\frac{\|b-Ax\|_2}{\|b\|_2}$ em todos os métodos.
- **Log por iteração (iterativos):** em cada passo k , grava-se $(k, R^{(k)}, x^{(k)})$. Ao final, matplotlib gera o gráfico $\log_{10}(R) \times k$ em .png.
- **Saída em arquivo:** cada teste produz um .txt com x final, erro, tempo e log completo. Ao final do main.py, gerar_sumario() consolida tudo em Markdown.
- **Teste:** Inicialmente é gerado um x aleatório, calcula-se Ax e usa-se esse valor como b para a solução do sistema. Como métrica adicional para o erro, o valor de x encontrado pode ser comparado ao valor de x gerado antes.

4 Matrizes de teste

Foram utilizadas doze matrizes do repositório Matrix Market, seis mal-condicionadas e seis bem-condicionadas.

4.1 Matrizes Mal-Condicionadas

A Tabela 2 apresenta suas principais características.

Tabela 2: Matrizes utilizadas nos experimentos.

Matriz	n	$\kappa(A)$	Tipo
bcsstk01	48	$8,8 \times 10^5$	SPD
bcsstk03	112	$6,8 \times 10^6$	SPD
bcsstk04	132	$2,3 \times 10^6$	SPD
494_bus	494	$2,4 \times 10^6$	SPD
nos6	675	$7,7 \times 10^6$	SPD diag. dominante
1138_bus	1138	$8,6 \times 10^6$	SPD

Todas as matrizes são SPD (autovalores positivos) e mal-condicionadas, cobrindo dois domínios de aplicação e três ordens de grandeza em n .

4.2 Matrizes Bem-Condicionadas

A Tabela 3 apresenta suas principais características.

Tabela 3: Matrizes utilizadas nos experimentos.

Matriz	n	$\kappa(A)$	Tipo
bfb62b	62	$1,7 \times 10^1$	Real simétrica indefinida
bcsstm02	66	$8,8 \times 10^0$	SPD diag. dominante
bcsstm22	138	$9,4 \times 10^2$	SPD diag. dominante
bfb398b	398	$2,1 \times 10^1$	Real simétrica indefinida
dwb512	512	$3,7 \times 10^0$	Real não simétrica diag. dominante
gr_30_30	900	$1,9 \times 10^2$	SPD fracamente diag. dominante

Entre as matrizes selecionadas, temos quatro matrizes diagonais dominantes, sendo duas matrizes diagonais, e duas matrizes não diagonais dominantes. Apesar de serem bem condicionadas, nem todas são SPD, ou seja, o método de Cholesky não é aplicável para todas.

5 Resultados

5.1 Matrizes Mal-Condicionadas

5.1.1 Métodos diretos

A Tabela 6 apresenta os tempos de execução para os três métodos diretos. O erro residual $\frac{\|b-Ax\|_2}{\|b\|_2}$ ficou na faixa de 10^{-16} a 10^{-15} em todos os casos — solução essencialmente exata para os fins práticos.

Tabela 4: Tempo de execução (segundos) dos métodos diretos.

Matriz	Gauss	LU	Cholesky
bcsstk01	0,003	0,003	0,003
bcsstk04	0,059	0,057	0,040
494_bus	3,19	2,95	1,75
nos6	8,18	7,60	4,47
1138_bus	39,32	36,56	21,59

Cholesky levou aproximadamente 55% do tempo de Gauss/LU na maior matriz, em acordo com a razão teórica $\frac{n^{3/3}}{2n^{3/3}} = \frac{1}{2}$.

5.1.2 Métodos iterativos

A Tabela 7 apresenta os resultados de Jacobi e Gauss-Seidel com $t = 10^{-8}$ e $o = 4000$ iterações máximas, cada um com seu respectivo erro $\frac{\|b-Ax\|_2}{\|b\|_2}$.

Tabela 5: Resultados dos métodos iterativos ($t = 10^{-8}$, $o_{\max} = 4000$).

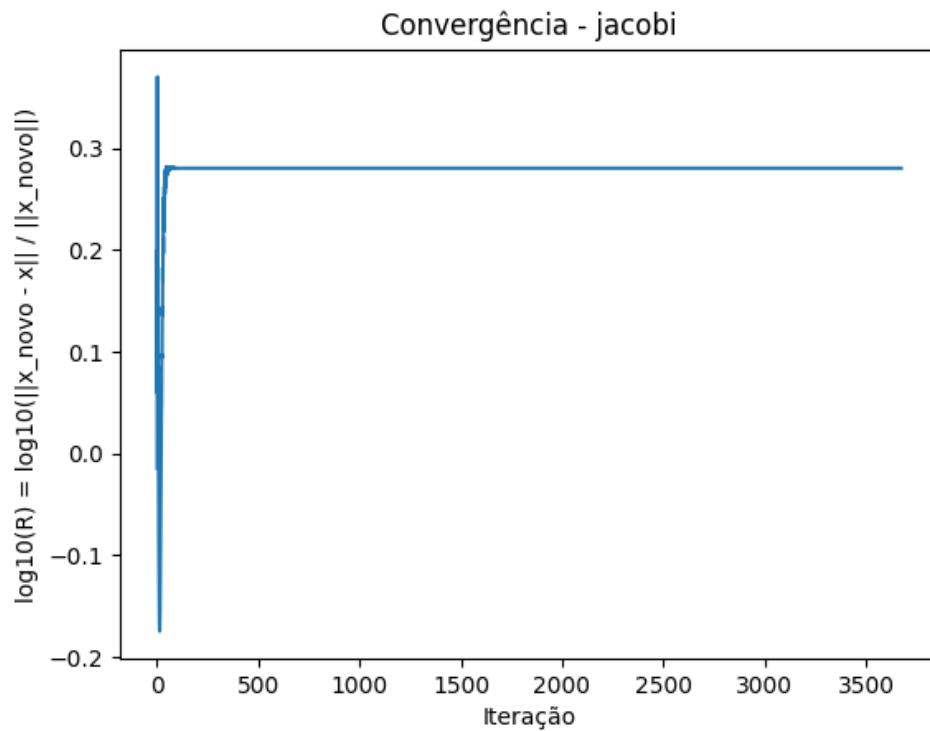
Matriz	Jacobi	Gauss-Seidel
bcsstk01	4000 iter — erro <i>INF</i>	3967 iter — $R < t$ (erro $1,65 \times 10^{-11}$)
bcsstk03	4000 iter — erro <i>INF</i>	4000 iter — erro $3,10 \times 10^{-5}$
bcsstk04	4000 iter — erro <i>INF</i>	4000 iter — erro $3,03 \times 10^{-3}$
494_bus	4000 iter — erro 0,44	4000 iter — erro $3,42 \times 10^{-5}$
nos6	4000 iter — erro 0,13	4000 iter — erro $8,23 \times 10^{-7}$
1138_bus	4000 iter — erro 0,21	4000 iter — erro $2,51 \times 10^{-5}$

5 das matrizes utilizadas não são diagonais dominantes, ou seja, não há garantia de convergência para elas. Apesar disso, na matriz bcsstk01, o método de Gauss-Seidel acabou convergindo com 3967 iterações. Além disso, apesar da nos6 ser diagonal dominante, por ser mal condicionada, ela não convergiu dentro do limite da quantidade máxima de iterações do método de Jacobi.

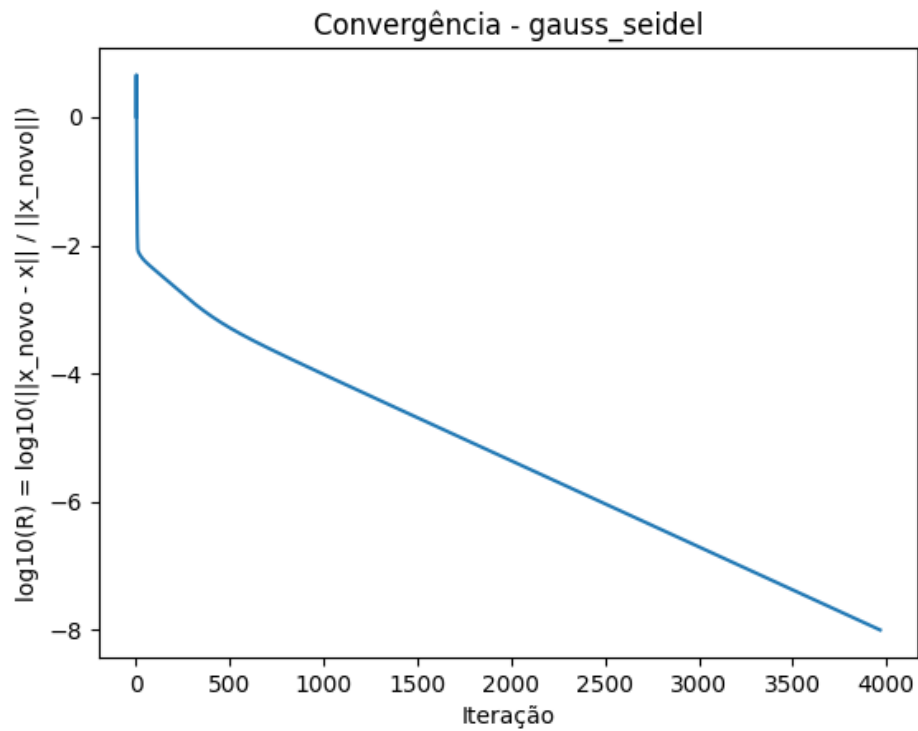
Além do mais, devido a má condicionalidade, o método de Jacobi apresentou um grande erro relativo para todas as matrizes. O método de Gauss-Seidel apresentou erro relativo menor.

5.1.3 Gráficos $\log(r)$ x iterações

- Matriz bcsstk01

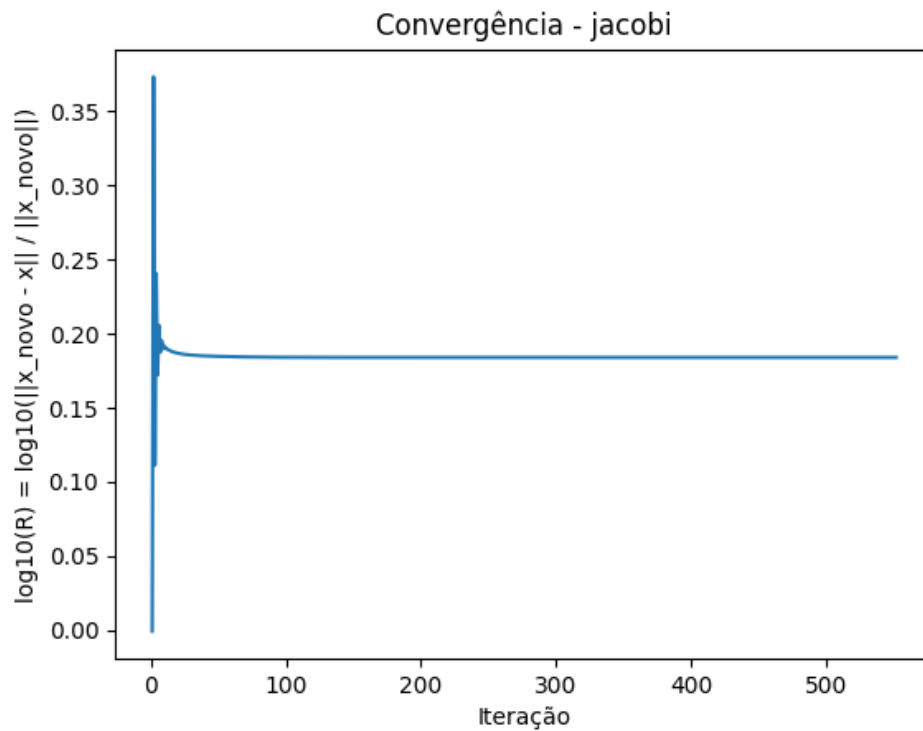


O gráfico de Jacobi acabou explodindo, demonstrando que para matrizes não diagonais dominantes a convergência não é garantida.

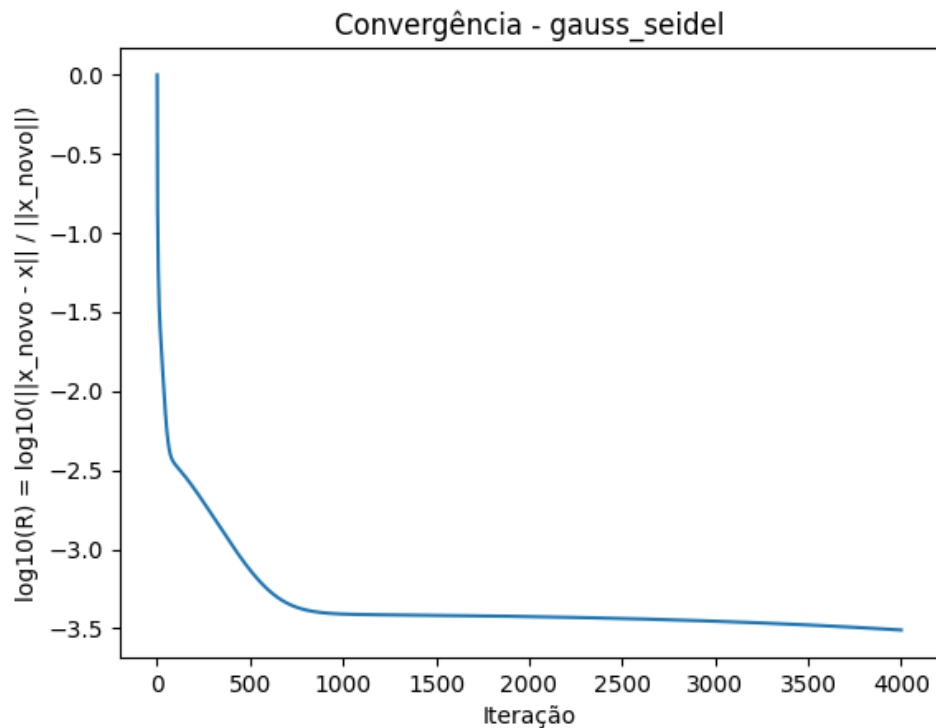


O gráfico de Gauss-Seidel demonstrou um comportamento decrescente e finalizou com 3967 operações apresentando um erro relativo $1,65 \times 10^{-11}$.

- Matriz bcsstk03

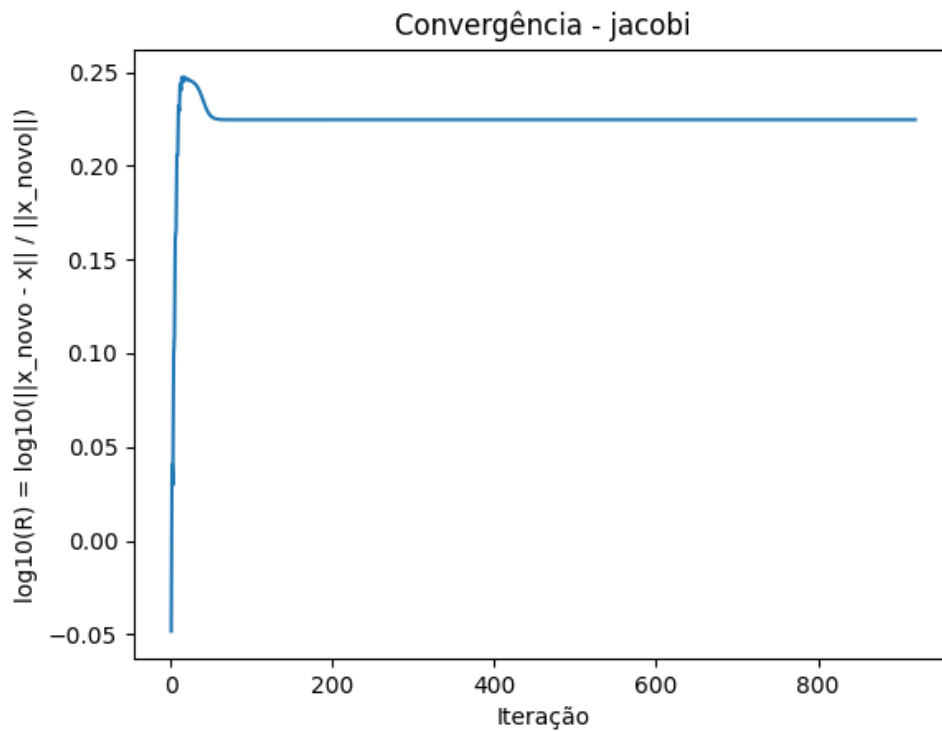


O gráfico de Jacobi acabou explodindo, demonstrando que para matrizes não diagonais dominantes a convergência não é garantida.

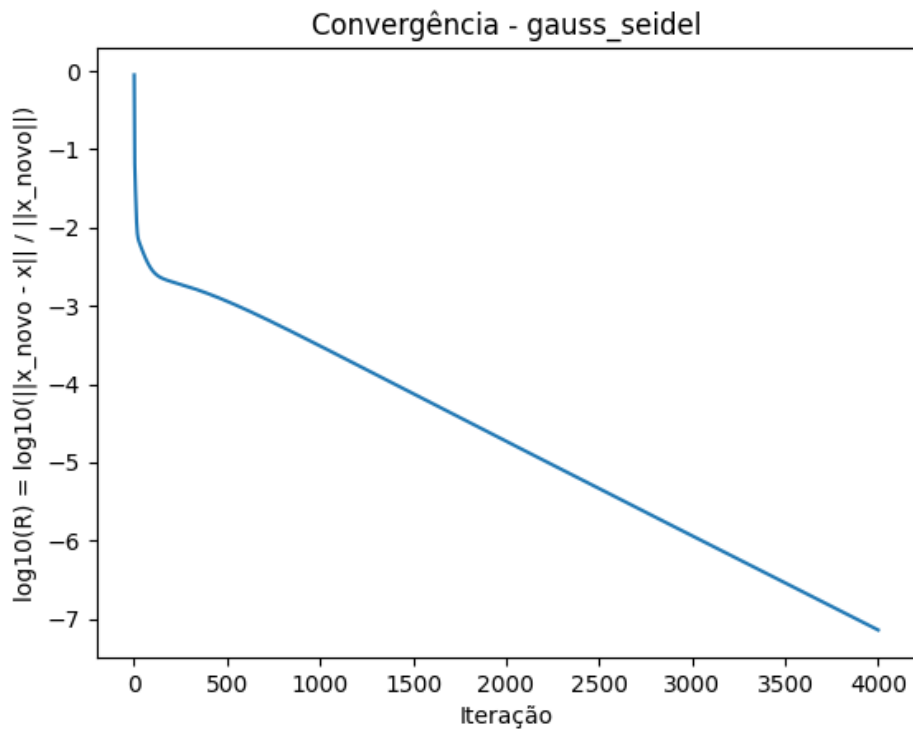


O gráfico de Gauss-Seidel demonstrou um comportamento decrescente, no entanto, a partir de 1000 operações começou a ter um comportamento constante.

- Matriz bcsstk04

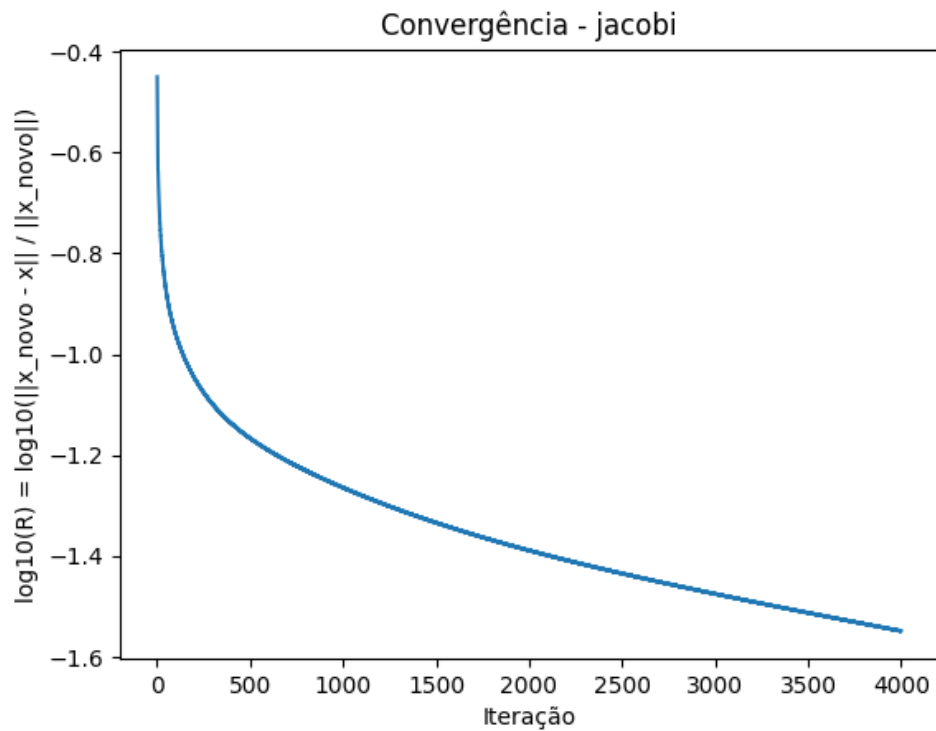


O gráfico de Jacobi acabou explodindo, demonstrando que para matrizes não diagonais dominantes a convergência não é garantida.

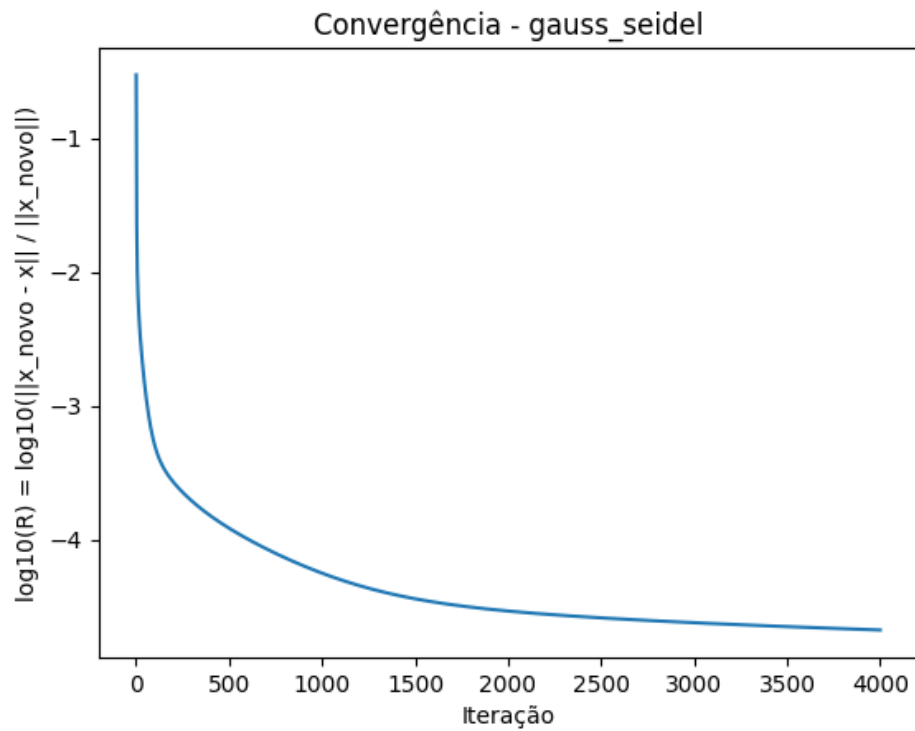


O gráfico de Gauss-Seidel demonstrou um comportamento decrescente, no entanto, não convergiu dentro do limite de operações máximas.

- Matriz 494_bus

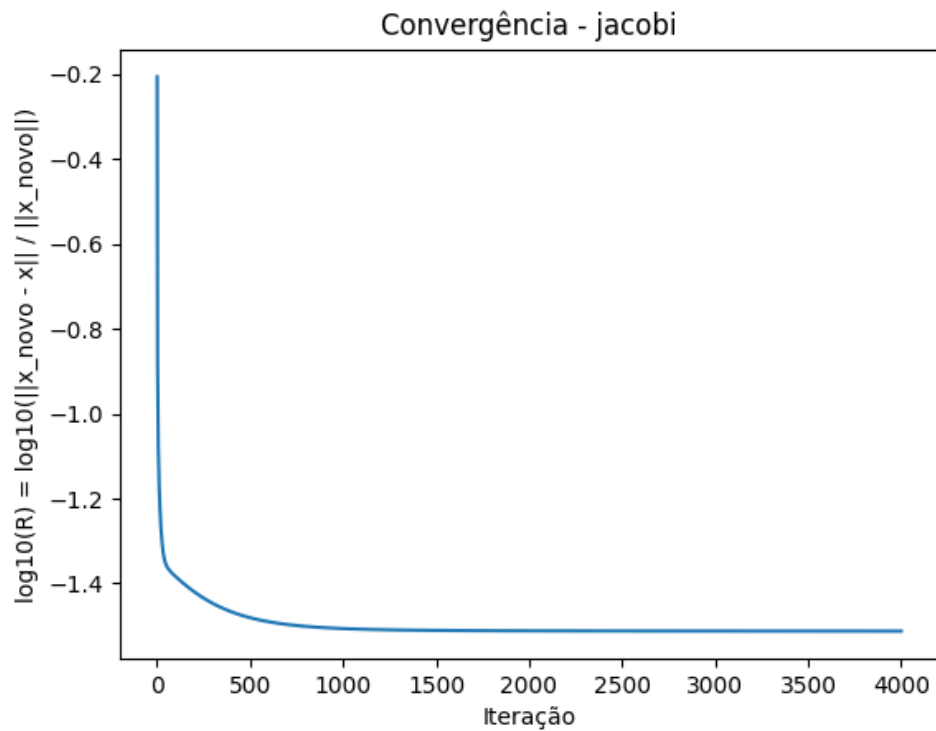


Esse gráfico de Jacobi demonstrou um comportamento decrescente, no entanto, não convergiu dentro do limite de operações máximas.

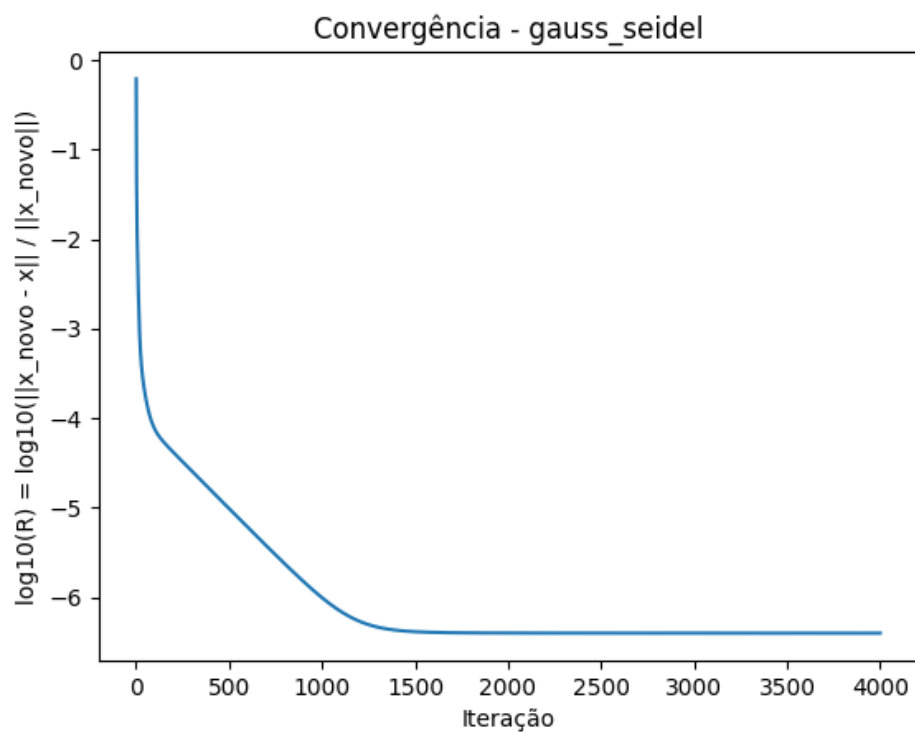


O gráfico de Gauss-Seidel demonstrou um comportamento decrescente, no entanto, não convergiu dentro do limite de operações máximas.

- Matriz nos6

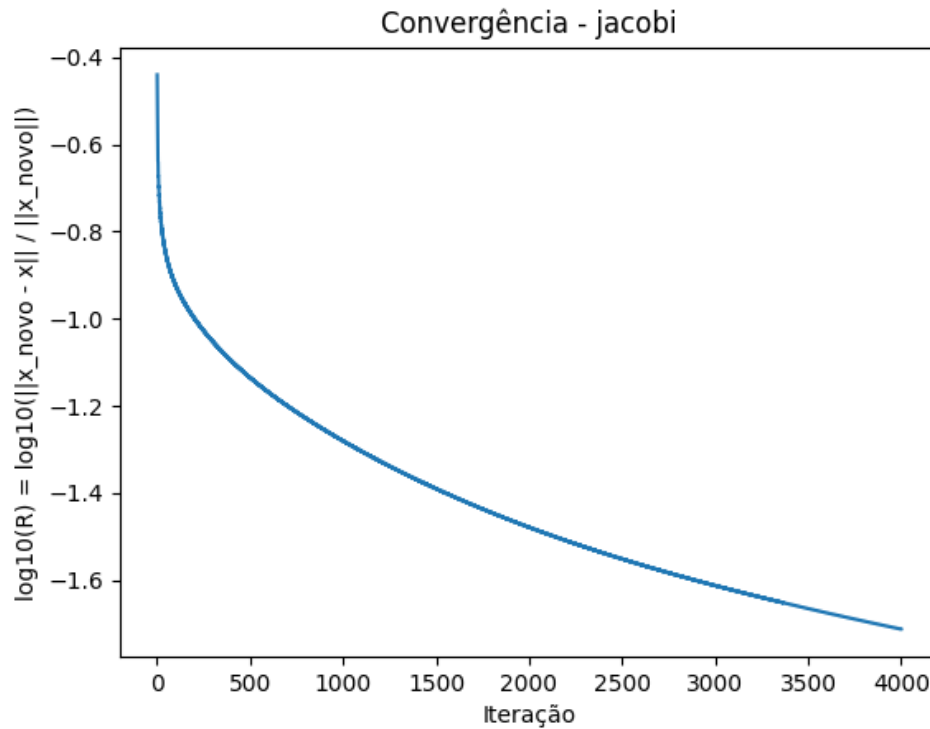


Esse gráfico de Jacobi demonstrou um comportamento decrescente até a milésima iteração, no entanto, entrou em um comportamento constante e não convergiu no limite máximo de iterações. Apesar da matriz ser diagonal dominante, os métodos não convergiram no limite máximo de iterações, o que demonstra como a má condicionalidade pode prejudicar a convergência dos métodos iterativos.

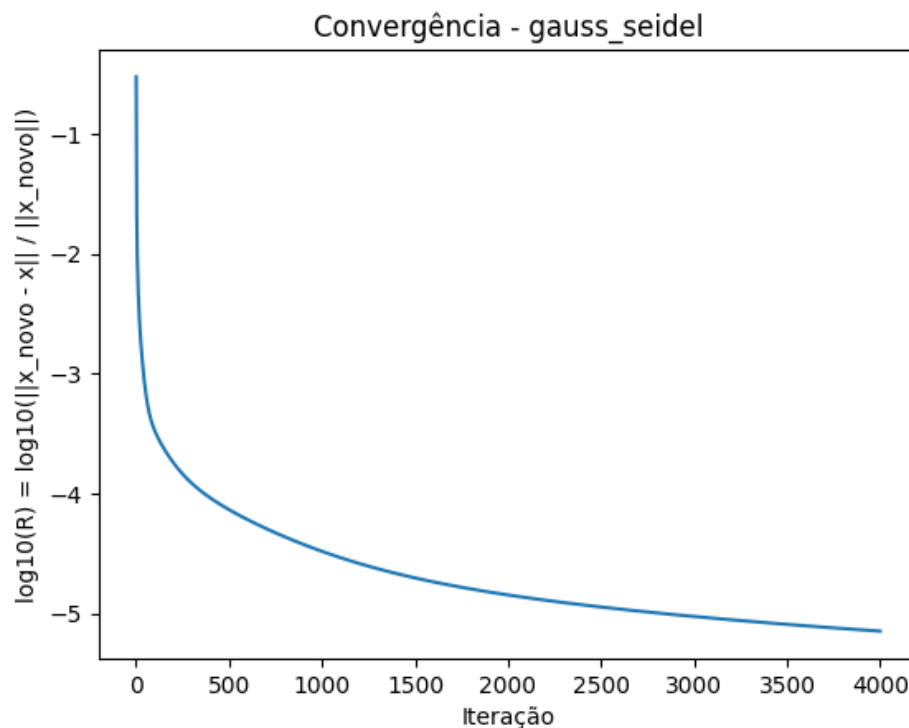


Esse gráfico de Gauss-Seidel demonstrou um comportamento decrescente até a iteração 1500, no entanto, entrou em um comportamento constante e não convergiu no limite máximo de iterações. Apesar da matriz ser diagonal dominante, os métodos não convergiram no limite máximo de iterações, o que demonstra como a má condicionalidade pode prejudicar a convergência dos métodos iterativos.

- Matriz 1138_bus



O gráfico de Jacobi obteve comportamento decrescente, mas não convergiu no limite máximo de iterações.



O gráfico de Gauss-Seidel obteve comportamento decrescente, mas não convergiu no limite máximo de iterações.

5.2 Matrizes Bem-Condicionadas

5.2.1 Métodos diretos

A Tabela 6 apresenta os tempos de execução para os três métodos diretos. O erro residual $\frac{\|b - Ax\|_2}{\|b\|_2}$ ficou na faixa de 10^{-16} a 10^{-15} em todos os casos (à exceção do método de Cholesky nas matrizes que não são SPD) — solução essencialmente exata para os fins práticos.

Tabela 6: Tempo de execução (segundos) dos métodos diretos.

Matriz	Gauss	LU	Cholesky
bfw62b	0,012	0,012	-
bcsstm02	0,018	0,009	0,006
bcsstm22	0,105	0,098	0,093
bfw398b	2,01	2,39	-
dwb512	6,64	5,89	-
gr_30_30	23,59	29,41	14,50

Cholesky levou aproximadamente 55% do tempo de Gauss/LU na maior matriz, em acordo com a razão teórica $\frac{n^{3/3}}{2n^{3/3}} = \frac{1}{2}$.

5.2.2 Métodos iterativos

A Tabela 7 apresenta os resultados de Jacobi e Gauss-Seidel com $t = 10^{-8}$ e $o = 2000$ iterações máximas, cada um com seu respectivo erro $\frac{\|b-Ax\|_2}{\|b\|_2}$.

Tabela 7: Resultados dos métodos iterativos ($t = 10^{-8}$, $o_{\max} = 2000$).

Matriz	Jacobi	Gauss-Seidel
bfw62b	2000 iter — erro $1,26 \times 10^7$	19 iter — $R < t$ (erro $7,64 \times 10^{-10}$)
bcsstm02	2 iter — $R < t$ (erro 0)	2 iter — $R < t$ (erro 0)
bcsstm22	2 iter — $R < t$ (erro 0)	2 iter — $R < t$ (erro 0)
bfw398b	2000 iter — erro $1,28 \times 10^{43}$	19 iter — $R < t$ (erro $9,31 \times 10^{-10}$)
dwb512	16 iter — $R < t$ (erro $5,04 \times 10^{-10}$)	9 iter — $R < t$ (erro $4,46 \times 10^{-10}$)
gr_30_30	1723 iter — $R < t$ (erro $1,78 \times 10^{-8}$)	908 iter — $R < t$ (erro $8,91 \times 10^{-9}$)

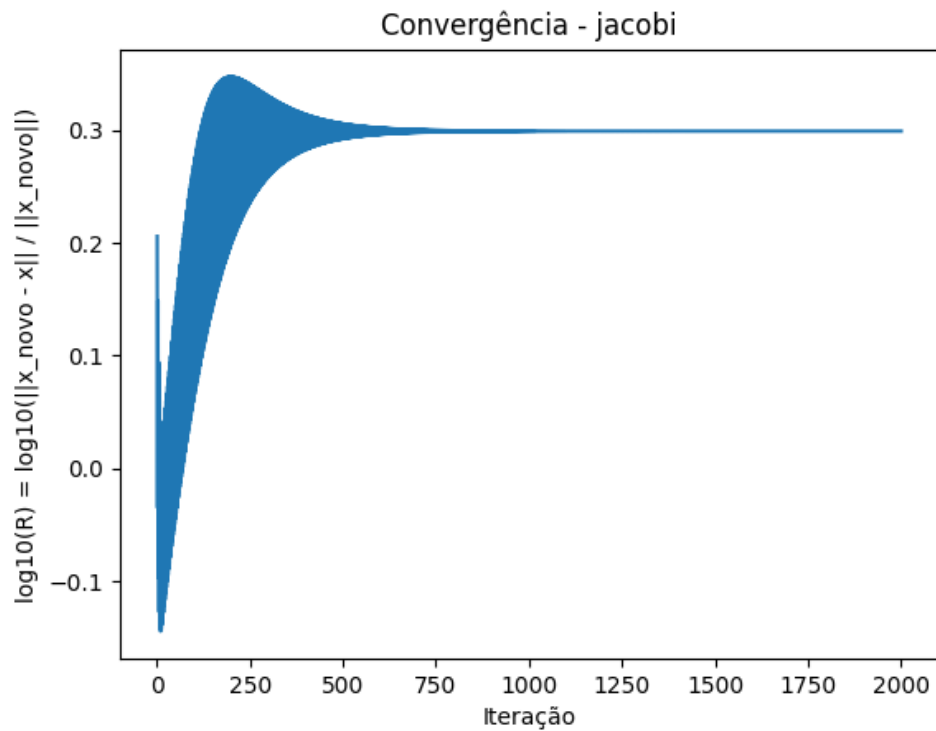
Neste momento, analisando as matrizes bem condicionadas, denota-se que, à exceção do método de Jacobi na matriz bfw398b, o erro obtido foi menor para todas as matrizes.

Além disso, devido a boa condicionalidade, o método de Gauss-Seidel convergiu em todas matrizes, mesmo nas matrizes não diagonais dominantes.

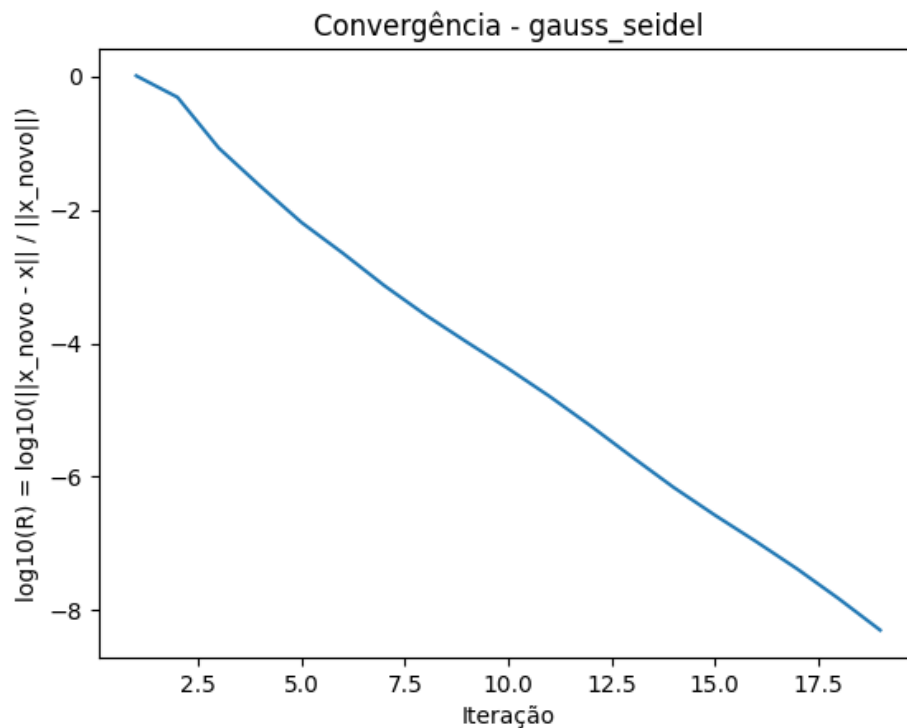
O erro 0 obtido e as 2 iterações realizadas devem se às matrizes bcsstm02 e bcsstm22 serem matrizes diagonais.

5.2.3 Gráficos $\log(r)$ x iterações

- Matriz bfw62b

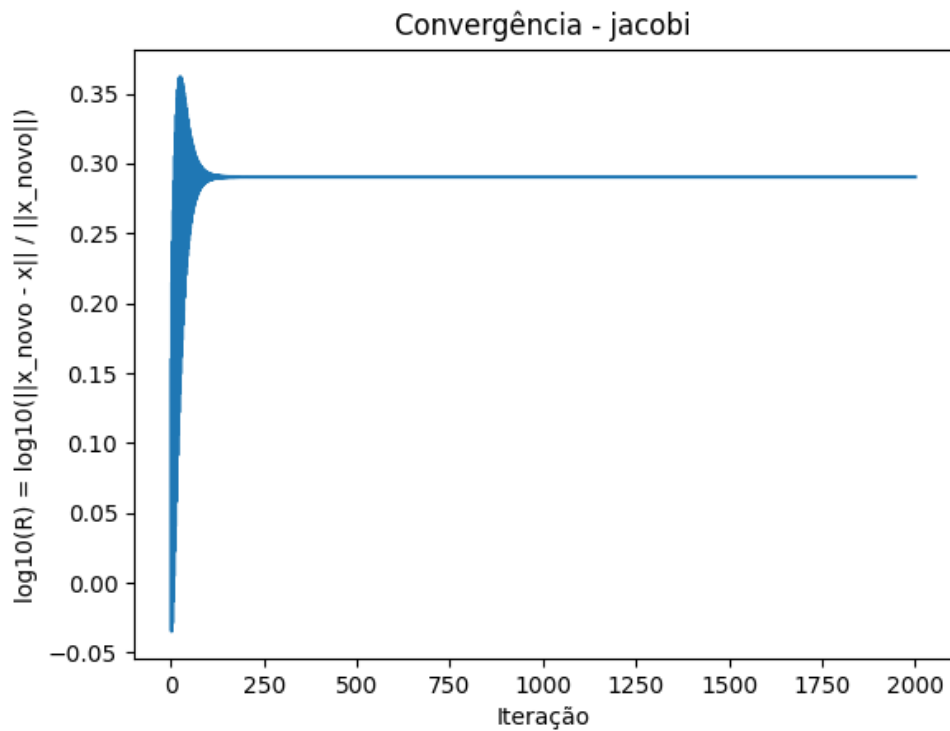


O Gráfico de Jacobi divergiu, não obtendo a solução correta.

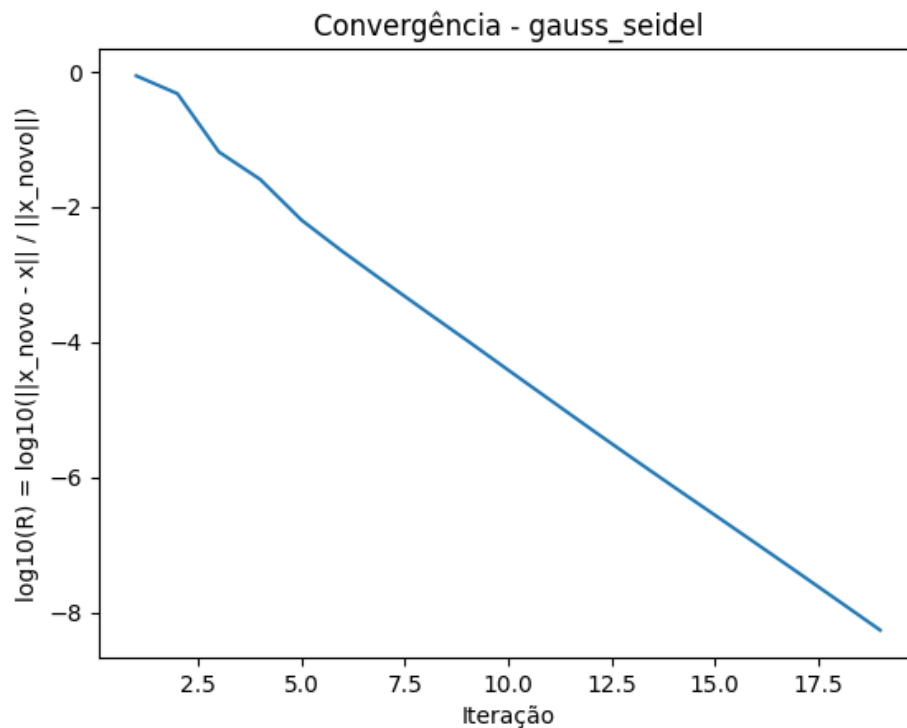


O Gráfico de Gauss-Seidel apresentou um comportamento de decrescimento, praticamente, linear e convergiu em 19 iterações.

- Matriz bfw398b

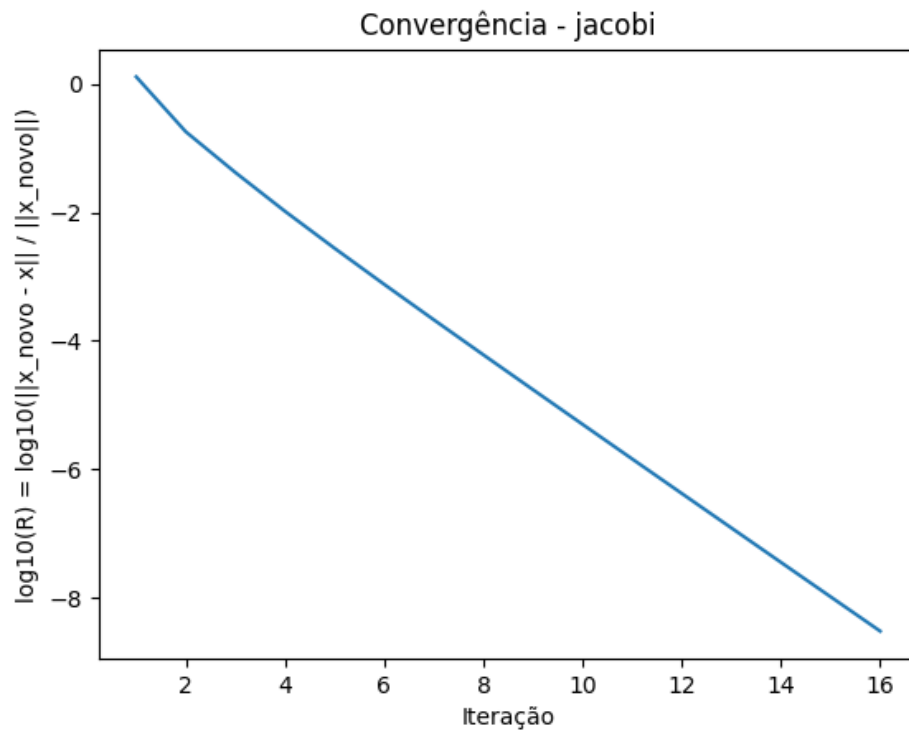


O Gráfico de Jacobi divergiu, não obtendo a solução correta.

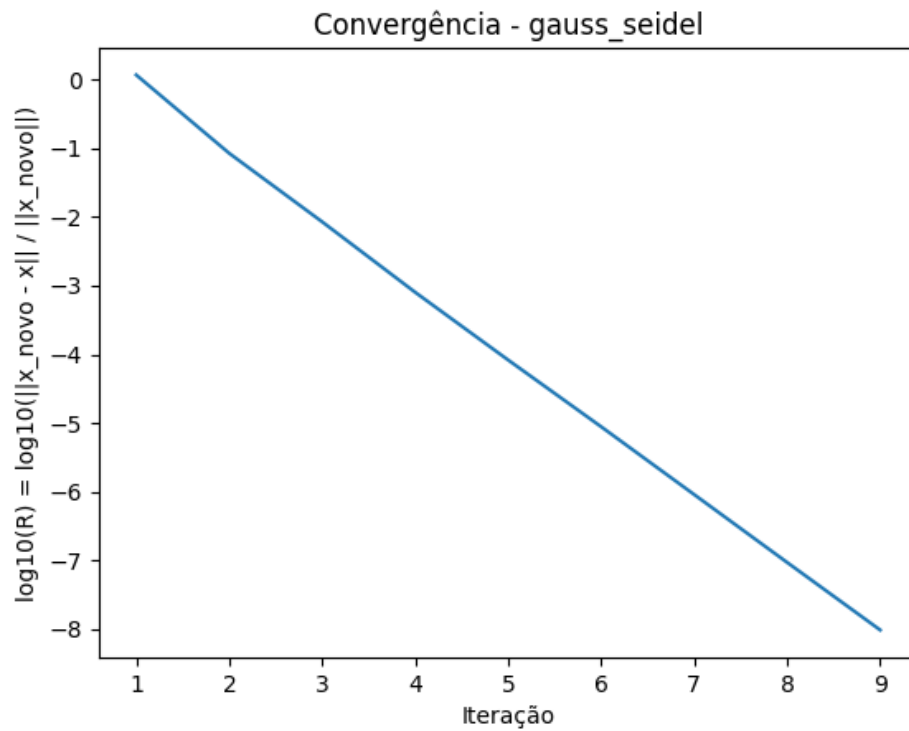


O Gráfico de Gauss-Seidel apresentou um comportamento de decrescimento, praticamente, linear e convergiu em 19 iterações.

- Matriz dwb512

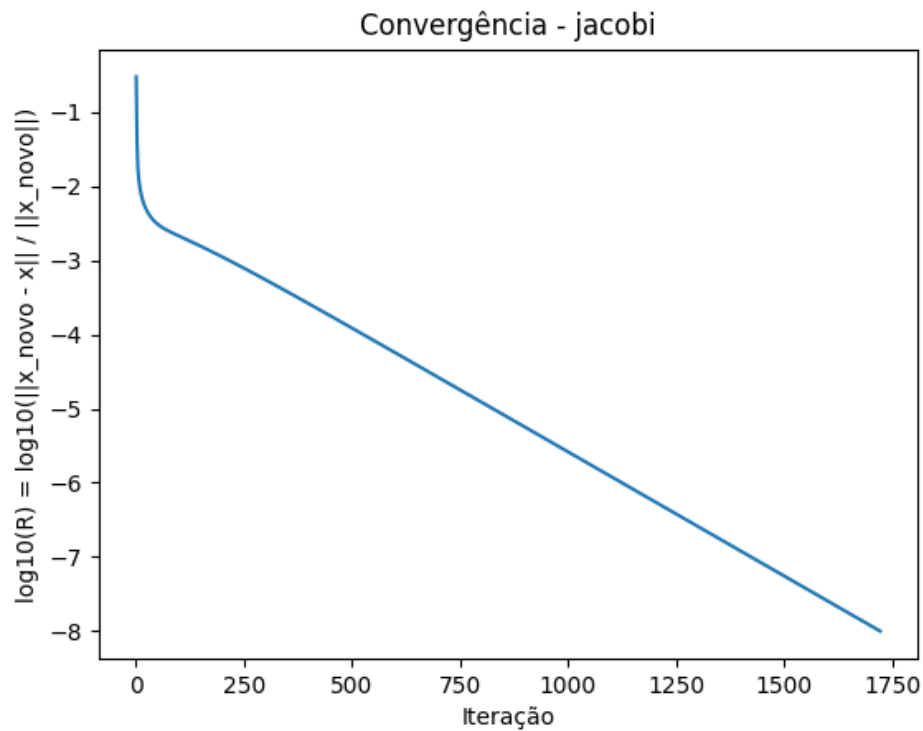


O gráfico de Jacobi apresentou um comportamento de decrescimento, praticamente, linear e convergiu em 16 iterações.

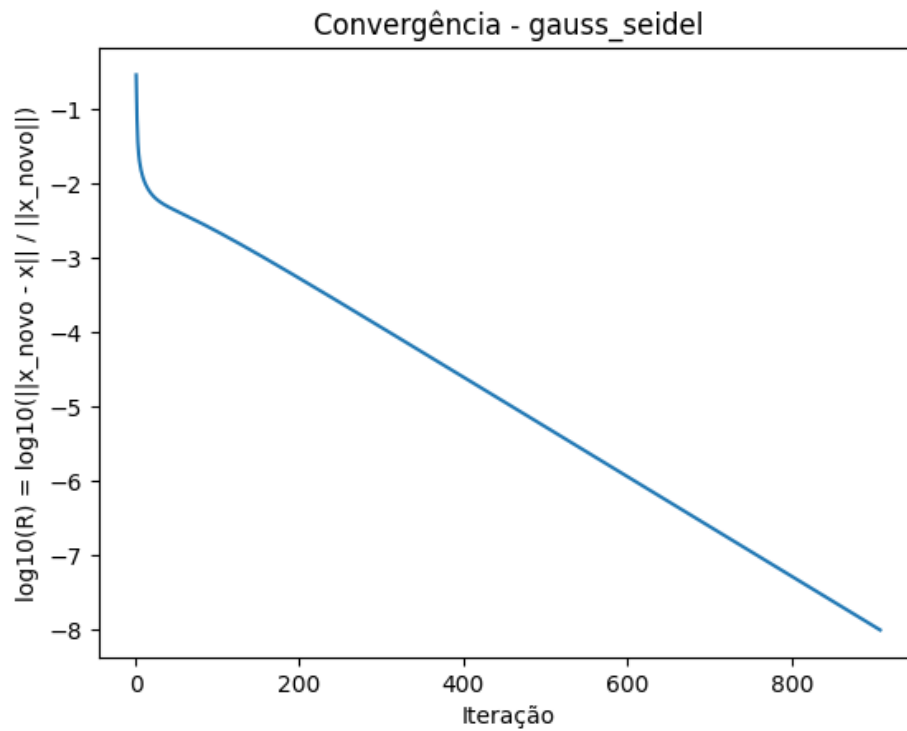


O Gráfico de Gauss-Seidel apresentou um comportamento, de decrescimento, praticamente, linear e convergiu em 9 iterações.

- Matriz gr_30_30



O gráfico de Jacobi apresentou um comportamento, a partir de 250 iterações, de decrescimento, praticamente, linear e convergiu em 1723 iterações.



O Gráfico de Gauss-Seidel apresentou um comportamento, a partir de 200 iterações, de decrescimento, praticamente, linear e convergiu em 908 iterações.

As matrizes bcsstm02 e bcsstm22 não geraram gráficos, por terem terminado em duas

iterações.

O método de Gauss-Seidel convergiu em todas as matrizes bem-condicionada, mesmo as que não eram diagonais dominantes.

Verifica que em matrizes bem-condicionadas, o gráfico de $\log(\text{resíduo}) \times \text{iteração}$ tende a ser linear.

6 Comparação: implementação pura vs. bibliotecas

6.1 Tempo de execução

A Tabela 8 apresenta os tempos e ganhos para a maior matriz (1138_bus, $n = 1138$).

Tabela 8: Tempo de execução e ganho de velocidade para 1138_bus.

Implementação	Tempo (s)	Ganho
Eliminação Gaussiana (Python puro)	39,3	1×
<code>np.linalg.solve</code>	0,062	632×
Fatoração LU (Python puro)	36,6	1×
<code>scipy.linalg.lu_solve</code>	0,479	76×
Cholesky (Python puro)	21,6	1×
<code>scipy.linalg.cho_solve</code>	0,069	313×

O ganho de velocidade provém de BLAS/LAPACK compilados em C/Fortran com instruções SIMD, em contraste com os laços Python interpretados.

6.2 Precisão numérica

Tabela 9: Erro residual $\frac{\|b-Ax\|_2}{\|b\|_2}$ em 1138_bus.

Método	Python puro	Biblioteca
Eliminação Gaussiana	$4,62 \times 10^{-16}$	$2,66 \times 10^{-16}$
Fatoração LU	$5,50 \times 10^{-16}$	$2,31 \times 10^{-16}$
Cholesky	$4,00 \times 10^{-16}$	$2,61 \times 10^{-16}$

A precisão é da mesma ordem (10^{-16}) entre a versão pura e as bibliotecas. A diferença é exclusivamente de eficiência computacional, não de algoritmo, confirmando a correção da implementação.

7 Validação da teoria: razão Cholesky/Gauss

A teoria prevê:

$$\frac{t_{\text{Cho}}}{t_{\text{Gauss}}} \longrightarrow \frac{n^3/3}{2n^3/3} = \frac{1}{2}. \quad (7)$$

A Tabela 10 mostra as razões medidas experimentalmente.

Tabela 10: Razão $t_{\text{Cho}}/t_{\text{Gauss}}$ por matriz.

Matriz	n	Razão medida
bcsstk03	112	0,63
bcsstk04	132	0,67
494_bus	494	0,55
nos6	675	0,55
1138_bus	1138	0,549

Para n grande, a razão converge ao valor teórico de 0,5, confirmando experimentalmente a previsão assintótica. Para matrizes pequenas, o overhead constante das implementações em Python afasta levemente o valor medido do limite teórico.

8 Conclusões

Os experimentos realizados permitem as seguintes conclusões:

1. **Cholesky é $\approx 2\times$ mais rápido que Gauss/LU** para matrizes SPD, em perfeito acordo com a previsão teórica $n^3/3$ vs. $2n^3/3$.
2. **Jacobi e Gauss-Seidel falharam em algumas das doze matrizes**, pois falta de diagonal dominância e mal condicionamento desfavorecem a convergência dos métodos iterativos clássicos. Para matrizes SPD mal-condicionadas, métodos diretos — especialmente Cholesky — são a escolha prática segura.
3. **Python puro vs. NumPy/SciPy**: a precisão numérica é muito próxima em ambas as abordagens; o ganho de velocidade de até $632\times$ das bibliotecas provém exclusivamente do uso de rotinas BLAS/LAPACK compiladas.

Reprodutibilidade

Para reproduzir todos os resultados basta executar:

```
python main.py
```

O script lê as seis matrizes em `Matrizes de Teste/`, roda os oito métodos em cada uma, gera os arquivos `.txt` e `.png` em `Resultados/<matriz>/` e produz o sumário

comparativo em `Resultados/sumario.txt`. O tempo total de execução é de aproximadamente 110 s na máquina de teste. Para adicionar uma nova matriz, basta depositar o arquivo `.mtx` em `Matrizes de Teste/` e incluí-lo na lista em `main.py`.